Zyra V1

A Content-Based Fashion Recommendation Engine Using Pretrained CLIP Embeddings, Cosine Similarity Retrieval, and Rule-Based Diversity Reranking

---

Weavly Engineering — Recommendation Systems Team

Engine Version: zyra-v1-p9 | Catalog Size: 12,465 Products | Embedding Dimension: 662

Document Type: Complete Technical Documentation
System: Production Recommendation Pipeline (Flask · Spring Boot · PostgreSQL)
Status: Production-Validated (P9 / P10 / R1 — All Passed)

Abstract

Zyra V1 is Weavly's first production recommendation system for fashion products. It is a **content-based recommendation engine** built on embeddings generated by a pretrained CLIP encoder — it is explicitly **not** a trained recommendation model, and no component was fine-tuned on user behavior. The system represents the full 12,465-product catalog as 662-dimensional vectors, retrieves candidates via cosine similarity, applies a hard gender-compatibility filter, computes a weighted relevance score across similarity, gender, category, brand, and price signals, and performs greedy diversity-aware reranking to produce a deterministic Top-50 recommendation set. The validated engine was packaged as a standalone Python service, exposed through a Flask inference API, integrated with a Spring Boot application layer, and connected to PostgreSQL for transactional, user-scoped recommendation persistence. This document consolidates the architecture, algorithms, production artifacts, validation history (P9, P10, R1), and the current system status, and clarifies the scope of what Zyra V1 does and does not know relative to a future behaviorally-trained V2.

Keywords: content-based recommendation, CLIP embeddings, cosine similarity, candidate retrieval, diversity reranking, Flask, Spring Boot, PostgreSQL, cold-start recommendation

Table of Contents

1. Executive Summary

Zyra V1 is Weavly's first production recommendation system for fashion products.

The most important clarification is stated below.

> ***Zyra V1 is not a trained recommendation model. It is a content-based recommendation engine powered by embeddings generated using a pretrained CLIP model.***

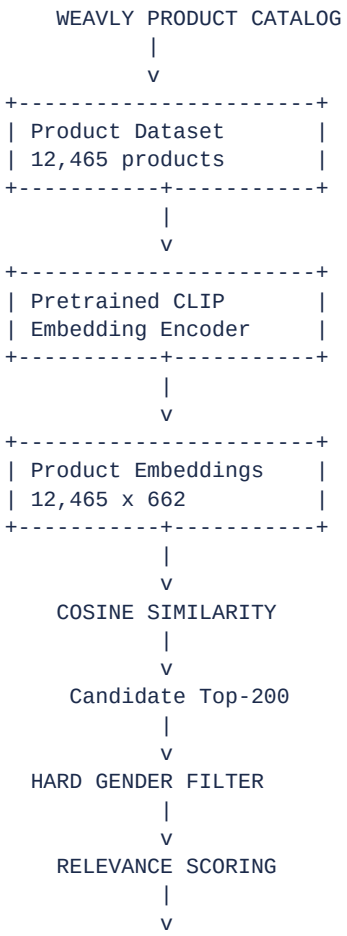
No recommendation model was fine-tuned or trained on user behavior in V1.

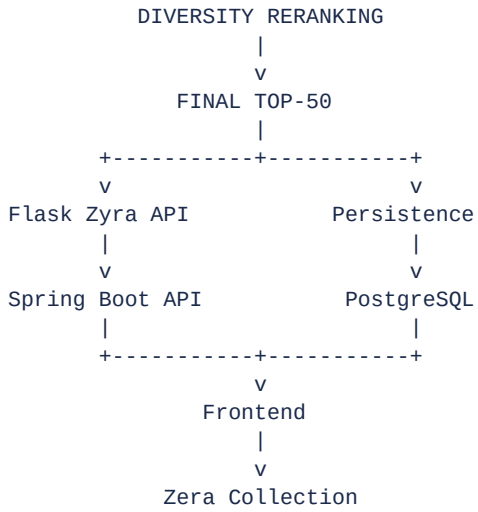
Instead, V1 takes the existing fashion catalog, represents products as numerical embeddings, finds visually/semantically similar products using **cosine similarity**, applies hard business constraints such as gender compatibility, calculates a relevance score, and finally applies diversity reranking to produce the final Top-50.

The production system was then packaged into a standalone Python engine, exposed through Flask, connected to Spring Boot, and connected to PostgreSQL for user recommendation persistence.

2. Zyra V1 Architecture

The complete V1 architecture is shown below.





3. What Was Actually Trained?

3.1 Recommendation Model

No recommendation model was trained in V1.

There was no:

- collaborative filtering training
- neural recommender training
- matrix factorization training
- LightFM training
- user-item interaction training
- ranking model training
- personalized neural network training
- reinforcement learning
- fine-tuning using user clicks/purchases

The V1 recommender is therefore **not a learned recommender**.

3.2 What Model Was Used?

A **pretrained CLIP model** was used as the embedding encoder.

Its job is not to directly recommend products. Its job is to convert products into numerical vector representations, conceptually:

```
   |
Embedding
   |
[0.021, -0.184, 0.392, ...]
```

The resulting production embedding matrix contains 12,465 products across 662 dimensions:

```
Embedding matrix = (12465, 662)
```

*The exact CLIP checkpoint/variant is **not established** by the documentation available here, so it should not be claimed as a specific CLIP variant without checking the original embedding-generation notebook.*

4. Product Embedding Layer

Every product is represented by a 662-dimensional vector. For example:

```
Product A -> [0.12, 0.44, -0.09, ..., 0.31]
Product B -> [0.11, 0.41, -0.07, ..., 0.29]
```

Products with similar semantic/visual characteristics tend to have embeddings that are closer together. This gives Zyra the foundation for content-based recommendation.

5. Cosine Similarity

The core retrieval mechanism is **cosine similarity**. For two product vectors A = [a_1, a_2, ..., a_n] and B = [b_1, b_2, ..., b_n], cosine similarity is defined as:

$$similarity(A, B) = (A \cdot B) / (\blacksquare A \blacksquare \blacksquare B \blacksquare)$$

The value approaches:

```
1.0  -> very similar
0.0  -> weak similarity
-1.0 -> opposite direction
```

The V1 embeddings were normalized. Production validation showed:

```
Mean L2 norm : 1.00000000
Minimum      : 0.99999994
Maximum      : 1.00000012
```

So cosine similarity can be computed efficiently using normalized vectors.

6. Candidate Retrieval

Zyra does not compare and directly return the final 50 products. The pipeline first retrieves a candidate pool of size K = 200:

```
Query product
      |
Cosine similarity against catalog
      |
Top 200 candidates
```

This creates a candidate pool large enough for subsequent business constraints and diversity processing.

7. Hard Gender Compatibility

One of the major V1 fixes was the **hard gender filter**. Initially, the embedding space could return semantically similar adult products for a Kids query — for example, a Kids jacket could retrieve a Women's jacket or a Men's jacket, because the embedding similarity was high. That was unacceptable for production. Therefore V1 introduced a hard compatibility layer **before** relevance scoring and diversity reranking.

7.1 Gender Normalization

Raw genders (Boys, Girls, Men, Women, Unisex, Unisex Kids) are normalized as follows:

Raw Label	Normalized Label
Boys	Kids
Girls	Kids
Unisex Kids	Kids
Men	Men
Women	Women
Unisex	Unisex

7.2 V1 Compatibility Rules

Query Gender	Eligible Catalog Genders
Women	Women + Unisex
Men	Men + Unisex
Unisex	Women + Men + Unisex
Kids	Kids only

This prevents adult products from entering Kids recommendations.

8. Relevance Scoring

After candidate retrieval and gender filtering, Zyra calculates the V1 relevance score. The frozen production weights are:

Component	Weight
Similarity	0.55

Component	Weight
Gender	0.20
Category	0.15
Brand	0.05
Price	0.05

Therefore the relevance score R is:

$$R = 0.55 \cdot S + 0.20 \cdot G + 0.15 \cdot C + 0.05 \cdot B + 0.05 \cdot P$$

where S = embedding similarity, G = gender compatibility score, C = category relationship, B = brand relationship, and P = price similarity.

These weights were **not learned from user behavior** — they are engineered V1 ranking weights.

9. Diversity Reranking

Pure similarity creates another problem. Suppose the query is *"SPYKAR Women's Jeans"*. Pure cosine similarity might return five near-identical SPYKAR jeans results in a row — technically similar, but poor recommendation diversity. Therefore V1 uses a greedy diversity reranking stage.

The configured diversity penalties are:

Occurrence	Penalty
1st occurrence	0.000
2nd occurrence	0.015
3rd occurrence	0.050
3+ occurrences	0.120

Conceptually, the reranking loop is:

```
Candidate score
      |
Check previously selected products
      |
Apply diversity penalty
      |
Select best remaining candidate
      |
Repeat
```

This allows Zyra to maintain similarity while preventing the Top-50 from becoming a collection of near-identical products.

10. Similarity Threshold

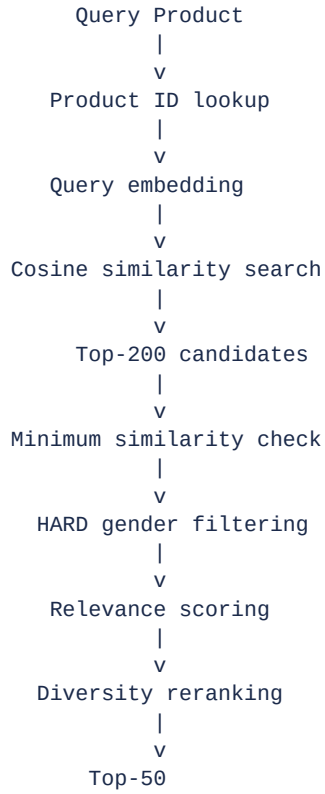
V1 uses a minimum similarity threshold:

```
Minimum similarity = 0.88
```

Recommendations below this threshold are rejected. The production contract therefore requires **similarity** ≥ **0.88**. During regression validation, the minimum observed similarity was **0.892401**.

11. Final Recommendation Pipeline

The final V1 algorithm is shown below. This ordering is important — the hard gender filter occurs **before** final scoring and diversity reranking.



12. Production Catalog

The V1 catalog contains 12,465 products at an embedding dimension of 662. The production artifacts are:

- product_embeddings.npy
- product_metadata.csv
- product_id_to_index.json
- zyra_v1_config.json
- manifest.json

13. Production Metadata

The production metadata contract contains:

```
productId
name
brand_clean
```

```
gender_clean
category_clean
price_numeric
```

The recommendation API additionally works with product presentation metadata such as imageUrl, productUrl, brand, gender, category, and price. This is particularly important for Weavly because the recommendation system must ultimately return something the frontend can display.

14. Production Artifacts

Artifact	Contents	Observed Size
product_embeddings.npy	12,465 × 662 normalized product vectors	31.48 MB
product_metadata.csv	Catalog metadata	1.06 MB
product_id_to_index.json	12,465 productId → embedding row mappings	—
zyra_v1_config.json	Frozen production configuration	—
manifest.json	Artifact manifest	—

The **zyra_v1_config.json** important values are:

```
{
  "engineVersion": "zyra-v1-p9",
  "candidateK": 200,
  "finalK": 50,
  "minimumSimilarity": 0.88,
  "embeddingDimension": 662
}
```

It also stores ranking weights, diversity penalties, gender compatibility, and catalog size.

15. Standalone Zyra Engine

The notebook implementation was converted into a standalone Python package with the following structure:

```
core-model/
|
+-- zyra/
|   +-- __init__.py
|   +-- config.py
|   +-- metadata.py
|   +-- engine.py
|   +-- persistence.py
|
+-- p10_production_artifacts/
|   +-- product_embeddings.npy
|   +-- product_metadata.csv
|   +-- product_id_to_index.json
|   +-- zyra_v1_config.json
|   +-- manifest.json
|
+-- app.py
```

```
+-- test_zyra_engine.py
+-- test_api.py
+-- test_persistence.py
+-- test_spring_integration.py
```

The major improvement was removing dependence on notebook/global state. The engine can now be initialized like a normal backend component.

16. ZyraV1 Engine

The standalone engine (**ZyraV1**) loads all production artifacts at startup. It validates:

- embedding dimensions
- product count
- metadata
- product IDs
- product ordering
- numerical integrity
- configuration
- compatibility rules

The engine does not contain database persistence logic. This separation is intentional.

17. Flask Production API

The standalone engine was exposed through Flask. Main endpoints:

```
GET  /health
GET  /info
POST /recommend
```

17.1 /health

Used for service health checking. Example response:

```
{
  "status": "ok",
  "service": "zyra-v1",
  "engineVersion": "zyra-v1-p9"
}
```

17.2 /info

Provides production engine information: products, embedding dimension, candidate K, final K, similarity threshold, and engine version.

17.3 /recommend

Accepts a product ID and top-K, and returns the recommendation collection:

```
{
  "productId": "10009781",
  "topK": 50
}
```

18. API Validation

The Flask API was tested across 19 representative products.

Metric	Result
Queries tested	19
Passed	19
Failed	0
Average latency	~112 ms

The API contract validated: exactly requested Top-K, no self recommendation, no duplicates, valid similarity, valid metadata, gender compatibility, deterministic ordering, and invalid ID handling.

19. PostgreSQL Persistence

The next stage connected recommendation generation with PostgreSQL. The architecture separates **Inference** from **Persistence**: the recommendation engine generates the recommendation, and a persistence service stores it.

20. Recommendation Generation Model

Two tables were introduced for recommendation persistence.

20.1 zyra_recommendation_generations

Stores the recommendation generation itself:

```
generation_id
query_product_id
model_version
item_count
generated_at
```

20.2 zyra_recommendation_items

Stores individual recommendations:

```
generation_id
query_product_id
recommended_product_id
rank
similarity
relevance_score
name
brand
gender
category
price
image_url
product_url
model_version
generated_at
```

21. Why Generation + Items?

Instead of storing a flat mapping of Product → [50 IDs], the system stores a Generation containing 50 ordered Recommendation items. This provides: historical tracking, ranking auditability, model-version tracking, reproducibility, immutable generations, and recommendation history.

22. Transactional Persistence

The recommendation generation and all 50 items are written transactionally:

```
BEGIN TRANSACTION
```

```
Create Generation

Insert Item 1
Insert Item 2
...
Insert Item 50

COMMIT
```

If an error occurs, the transaction is rolled back:

```
ROLLBACK
```

Therefore the database should never contain a Generation with only a partial set of items (e.g. 23/50). Persistence validation confirmed transaction atomicity.

23. Spring Boot Integration

Spring Boot became the application's main backend layer.

```
Frontend
   |
   v
Spring Boot
   |
   v
ZyraClient
   |
   v
Flask Zyra
   |
   v
ZyraV1
```

Spring Boot does not implement the recommendation algorithm. It acts as the application/API layer around Zyra.

24. Spring Boot Zyra Client

The Spring Boot backend contains **ZyraClient** and **ZyraClientImpl**. The client communicates with Flask using HTTP, and Spring Boot validates the response before returning it to the frontend. This creates a clean separation: Zyra is the ML/inference engine, and Spring Boot is the application/backend layer.

25. Spring Boot API

The public stateless product endpoint is:

```
GET /api/recommendations/product/{productId}
```

This allows the frontend to request recommendations for a product. The backend calls Flask's **POST /recommend** internally.

26. User Recommendation Architecture

The next layer associates recommendations with authenticated Weavly users. The important design decision is stated below.

> ***Spring Boot owns user identity.***

The authenticated JWT/Principal determines the user. The frontend does not tell the backend "userId = X" and expect that to be trusted. Instead:

```
JWT
 |
Spring Security
 |
Principal
 |
Authenticated User
 |
Recommendation Generation
```

This protects user isolation.

27. User Recommendation Persistence

The planned user-level architecture contains:

```
User
 |
 +-- Recommendation Generations
       |
       +-- Item 1
       +-- Item 2
       +-- ...
       +-- Item 50
```

This allows Zera to function as a view over the user's persisted recommendation generations rather than creating another redundant product collection.

28. Zera Collection Concept

For V1:

> ***Zera is a view over persisted recommendation generations.***

It does not need to duplicate the complete product catalog. The relationship is:

```
User
 v
Latest recommendation generation
 v
50 recommendation items
 v
Zera Collection
```

This preserves: history, generation IDs, model version, ranking, timestamps, and recommendation metadata.

29. Frontend Architecture

The frontend should consume Spring Boot rather than directly owning recommendation logic. The desired architecture is:

```
Next.js
   |
   v
Spring Boot
   |
   +-- PostgreSQL
   |      +-- User/catalog data
   |
   +-- Zyra Flask
          +-- Inference
```

The frontend does not calculate cosine similarity, does not rank products, and does not apply Zyra ranking weights.

30. Homepage Recommendation Section

The homepage should have normal catalog sections plus **one** Zyra recommendation section. The important distinction:

Section	Source
Normal Product Sections	Real catalog products
Zera Recommendations	Persisted Zyra recommendations

Zyra should not replace the entire homepage catalog.

31. Men's and Women's Pages

Gender filtering must remain strict at the frontend as a defensive layer:

Page	Eligible Genders
Men's	Men + Unisex
Women's	Women + Unisex
Kids	Kids

Even though Zyra already applies hard gender filtering, the frontend can defensively verify the returned data.

32. Real Product Data

The production system should use **real catalog data**. No fake products, fake prices, fake product names, or fake image URLs. The 12,465-product dataset is the foundation of the catalog, and the recommendation system should point to actual products from that catalog.

33. Product Images and URLs

The recommendation output needs product metadata such as productId, name, brand, price, imageUrl, and productUrl. The frontend should use the actual product image associated with each recommended product. Importantly, the recommendation engine itself should not be confused with an image-generation system — Zyra recommends **existing catalog products**; it does not generate new fashion images.

34. Why the Same Image Appeared Previously

The issue observed where multiple recommendation cards showed the same image was not evidence that cosine similarity was broken. It means the recommendation records/products being returned were associated with the same or incorrect image URL. There are two distinct layers:

```
Zyra:
"Recommend product IDs A, B, C, D..."

Catalog:
"A -> image A
 B -> image B
 C -> image C
 D -> image D"
```

If the catalog layer incorrectly maps all products to one image, the frontend will visually show duplicates even though Zyra returned different product IDs. Therefore image uniqueness is fundamentally a **catalog/product metadata integrity problem**, unless the dataset itself genuinely contains duplicate images.

35. Determinism

The V1 engine was explicitly tested for deterministic output. Repeated calls produced identical recommendation ordering. This is important because the same product, same catalog, and same configuration should produce the same result — making the system much easier to test, debug, deploy, reproduce, and audit.

36. Production Artifact Reload

The artifacts were tested from a fresh process rather than relying on notebook state. Validation included: embeddings reload, metadata reload, product index reload, configuration reload, product ordering, and embedding values — all passed. This was a critical step because it proved the engine wasn't only working because the notebook happened to have variables in memory.

37. Production Latency

The validated production engine performed very well for the current catalog size.

Stage	Average Latency
Standalone engine	~110 ms
Fresh artifact recommender	~108 ms
Production inference interface	~95 ms
Spring Boot integration	~114 ms

Exact latency varies between runs and environments. The key V1 quality gate was **< 1 second**, which was comfortably achieved.

38. V1 Validation History

38.1 P9 — Recommendation Engine

The recommendation algorithm was extensively validated across 19 representative products, with 19/19 passing. The engine validated: Top-50, gender compatibility, similarity threshold, self exclusion, duplicate exclusion, required fields, rank integrity, and latency.

39. P10 — Production Packaging

P10 moved the validated engine toward production, across six steps.

Step	Description	Result
1	Engine packaged (12,465 products, 662 dimensions)	—
2	Regression testing	19/19 passed; 100% gender compatibility; min similarity 0.892401
3	Production artifacts exported	—
4	Fresh-process reload	19/19 passed; avg latency ≈ 107.81 ms
5	Production inference interface	19/19 passed; avg latency ≈ 94.60 ms
6	Edge-case & robustness validation	All passed

Step 6 validated invalid product IDs; final_k values of 1, 5, 10, 25, and 50; JSON serialization; duplicate detection; self recommendation; similarity; and determinism.

40. R1 — Backend Productization

R1 moved Zyra from a validated notebook to a backend service.

Step	Description	Result
1	Standalone engine packaging	PASS
2	Flask inference API (/health, /info, /recommend)	PASS
4	Recommendation persistence	PASS
5	Spring Boot integration	PASS

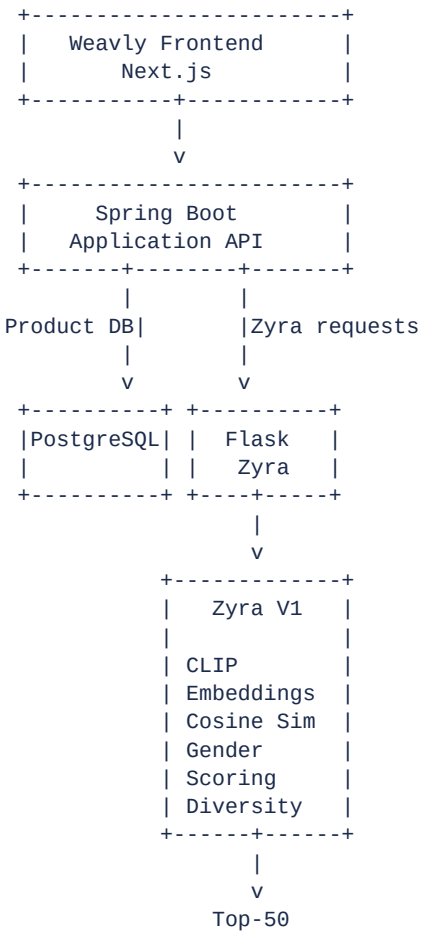
Step 1 validated: engine initialization, valid recommendation, Top-K, invalid ID, self exclusion, duplicate exclusion, similarity, gender, and determinism.

Step 4 validated: database connection, schema, generation creation, 50 items, rank preservation, similarity preservation, model version, no duplicates, self exclusion, atomicity, reload, and repeated generation.

Step 5 validated: Flask communication, DTO deserialization, response validation, Spring endpoint, error handling, timeout handling, a 19-product regression, 50 recommendations, ordering, and model version. Observed latencies: Flask ≈ 110.72 ms, Spring Boot ≈ 114.24 ms, integration overhead ≈ 3.52 ms.

41. Current Production Architecture

At the point reached so far, the architecture is essentially:



42. What Zyra V1 Is Good At

V1 is particularly good at **cold-start / content-based recommendation**. Given zero user/product interaction history, Zyra can still recommend products because it only needs the product representation. For example, given *"SPYKAR Women Blue Skinny Jeans"*, it can retrieve similar Women's jeans without needing thousands of users who previously bought the same item. That is a major advantage for the initial Weavly catalog.

43. What Zyra V1 Does NOT Know

Because V1 is not behaviorally trained, it does not inherently know statements such as:

- "This particular user likes black jeans."
- "This user always buys Nike."
- "This user dislikes skinny-fit clothing."
- "This user clicked these 20 products."
- "This user purchased these products."
- "This product converts better."

Those require behavioral/user interaction data. V1 primarily understands **Product ↔ Product similarity** rather than **User ↔ Product preference**.

44. V1 vs Future V2

This distinction is extremely important for Weavly.

44.1 V1

```
Product content -> CLIP embedding -> Cosine similarity -> Rules + scoring -> Top-50
```

44.2 V2 (potential)

```
User profile
      +
Product representation
      +
Clicks + Views + Wishlist + Cart + Purchases + Dwell time
      |
      v
Learned recommendation model
      |
      v
Personalized ranking
```

That would be a genuinely **trained recommender model**.

45. Why V1 Was the Correct First Step

For Weavly, jumping directly into a learned recommender would have a major problem: **there isn't enough user behavioral data yet**. A trained personalization system needs interaction data. V1 solves the cold-start problem — it provides a real catalog, immediate recommendations, and, as users interact with products, interactions accumulate into a training dataset that V2 can eventually learn from.

```
Real catalog
      |
Immediate recommendations
      |
Users interact with products
      |
Interactions accumulate
      |
Training dataset develops
      |
V2 can learn from those interactions
```

So V1 is effectively the foundation for the future recommendation-learning system.

46. Current Status

Component	Status
Product dataset	Complete — 12,465
Product embeddings	Complete
662-D representation	Complete
Cosine retrieval	Complete
Top-200 candidate retrieval	Complete
Hard gender filtering	Complete
Relevance scoring	Complete
Diversity reranking	Complete
Top-50 output	Complete
Similarity threshold	Complete — 0.88
Standalone engine	Complete
Production artifacts	Complete
Flask API	Complete
PostgreSQL persistence layer	Complete
Spring Boot integration	Complete
User recommendation architecture	Complete
Frontend consumption	In integration
Production catalog population	Current integration task
Behavioral personalization	Not in V1
Trained recommender	Not in V1

47. The Most Accurate Technical Description

For documentation, resume, GitHub, or architecture presentation purposes, the most accurate description of Zyra V1 is as follows.

> ***Zyra V1 is a production-oriented content-based fashion recommendation engine that uses embeddings generated from a pretrained CLIP encoder to represent 12,465 catalog products in a 662-dimensional vector space. It retrieves candidates using cosine similarity, applies hard gender compatibility filtering, computes a weighted relevance score using similarity, gender, category, brand, and price signals, and performs diversity-aware reranking to produce a deterministic Top-50 recommendation set. The engine is packaged as a standalone Python inference service, exposed through Flask, integrated with Spring Boot, and designed for transactional PostgreSQL persistence and authenticated user-specific Zera collections.***

And the critical wording to preserve is:

> ***Pretrained CLIP encoder — not a trained recommendation model.***

That is the technically honest description of everything built in Zyra V1 to date.

End of Document — Zyra V1 Technical Documentation, Weavly Engineering.